

F.CSM311 — Бие даалт 13

AI-Assisted Software Construction

URL Shortener — REST API + Minimal Frontend

B232270032

2026 оны 5 сарын 10

Аннотация

Энэхүү тайланд URL Shortener буюу урт URL холбоосыг богино холбоос болгон хувиргах REST API болон minimal frontend бүхий төслийн төлөвлөлт, хэрэгжүүлэлт, AI ашигласан workflow, гарсан асуудал, reflection хэсгийг нэгтгэн тайлбарлав. Төсөл нь Node.js, Express.js, SQLite, plain HTML/CSS/JavaScript, Jest + Supertest, OpenAPI 3.0 зэрэг технологи дээр суурилсан. AI assistant-ийг project planning, architecture review, documentation draft, test idea, security checklist зэрэгт ашигласан боловч final шийдвэр, алдаа шалгах, Git болон terminal output унших, screenshot evidence бүрдүүлэх ажлыг хүний review-тэй хийсэн.

Содержание

1 Оршил	4
2 Төслийн товч танилцуулга	4
2.1 Хэрэгжүүлсэн feature-үүд	4
2.2 Ашигласан технологи	5
3 Part A — Төлөвлөлт	5
3.1 Project brief ба scope	5
3.2 Stack comparison	5
3.3 Architecture diagram	6
3.4 Repository structure	6
3.5 Setup screenshot evidence	7
3.6 ADR-001: Stack decision	10
4 Part B — Build	10
4.1 Implementation result	10
4.2 API endpoint-үүд	12
4.3 Test plan	13
4.4 OpenAPI 3.0	13
4.5 Custom slash commands	14
4.6 Commit workflow	14
5 Part C — Reflection	14
5.1 AI-г юунд ашигласан, юуг өөрөө хийсэн бэ?	14
5.2 Hallucination жишээ ба зассан арга	15
5.3 Security болон license-ийн анхаарал	15
5.4 AI юуг хурдан болгосон бэ?	16
5.5 AI юуг удаан болгосон бэ?	16

5.6 Skill atrophy эрсдэлийг хэрхэн зохицуулсан бэ?	16
6 ADR-002: Storage decision	17
7 Өөрийн үнэлгээ	17
7.1 Хэрэв шалгалт өнөөдөр болбол би энэ кодыг өөрөө бичиж чадах уу?	17
7.2 Дахин хийнэ гэвэл юуг өөрөөр хийх вэ?	17
7.3 Энэ туршлагаас юу сурсан бэ?	17
8 Дүгнэлт	18

1 Оршил

Энэ тайлан нь F.CSM311 хичээлийн **Бие даалт 13: AI-Assisted Software Construction** даалгаварт зориулсан project documentation юм. Миний сонгосон project бол **URL Shortener — REST API + minimal frontend**. Энэ project-ийн үндсэн зорилго нь хэрэглэгчийн оруулсан урт URL холбоосыг богино code бүхий URL болгон хадгалж, дараа нь богино URL дээр дарахад original URL руу redirect хийх систем боловсруулах явдал байсан.

Даалгаврын гол санаа нь зөвхөн code бичих биш, харин AI assistant-ийг software construction workflow-д хэрхэн ашигласнаа баримтжуулах явдал гэж ойлгосон. Тиймээс энэ тайланд project-ийн төлөвлөлт, stack сонголт, architecture, repository structure, build feature-үүд, OpenAPI, test plan, AI session log, hallucination жишээ, security/license анхаарал, өөрийн үнэлгээ зэрэг хэсгүүдийг нэг дор цэгцтэй оруулсан.

Тайланг Монгол хэлээр, бодит оюутны тайлангийн хэв маягтай, хэт төгс хиймэл биш байдлаар бичихийг зорьсон. Мөн “бүх test pass болсон” гэх мэт бодитоор ажиллуулж нотлогүй claim оруулаагүй. Test-ийн талаар `npm test` command-аар ажиллуулах зориулалттай, ажиллуулсны дараа үр дүнг тэмдэглэх ёстой гэж үнэнээр тусгасан.

2 Төслийн товч танилцуулга

URL Shortener систем нь дараах үндсэн flow-той:

1. Хэрэглэгч frontend form дээр урт URL оруулна.
2. Backend API random short code үүсгэнэ.
3. Original URL, short code, click count, created date, optional expiration date зэргийг SQLite database-д хадгална.
4. Хэрэглэгч short URL нээхэд system original URL руу redirect хийнэ.
5. Redirect бүрийн дараа click count нэмэгдэнэ.
6. Statistics endpoint-оор тухайн short URL-ийн мэдээллийг харж болно.

2.1 Хэрэгжүүлсэн feature-үүд

№	Feature	Тайлбар
1	Short URL үүсгэх	Урт URL оруулахад random short code үүсгэж database-д хадгална.
2	Redirect хийх	<code>/:code</code> endpoint-оор original URL руу шилжүүлнэ.

3	Click count	Богино холбоос дарагдсан тоог бүртгэнэ.
4	Expiration validation	Optional хугацаа дуусах огноо байгаа бол шалгана.
5	Statistics endpoint	Short URL-ийн original URL, click count, created date зэрэг мэдээллийг харуулна.
6	Minimal frontend	HTML/CSS/JavaScript form ашиглан API-г browser дээрээс турших боломжтой.

2.2 Ашигласан технологи

Давхарга	Технологи
Backend	Node.js, Express.js
Database	SQLite
Frontend	Plain HTML/CSS/JavaScript
Test	Jest, Supertest
API documentation	OpenAPI 3.0 YAML
Version control	Git, GitHub
AI workflow	Prism / AI assistant

3 Part A — Төлөвлөлт

3.1 Project brief ба scope

Project-ийн scope-ийг жижиг боловч бүрэн ажиллах REST API project байхаар тогтоосон. Үүнд long URL-ээс short URL үүсгэх, short code-оор redirect хийх, click count хадгалах, expiration date шалгах, statistics endpoint raprax, minimal frontend хийх feature-үүд орсон. Харин authentication, user dashboard, payment, analytics chart, deployment, admin panel зэрэг том feature-үүдийг scope-оос хассан.

Энэ decision-ийн шалтгаан нь даалгаврын гол зорилго AI-assisted construction workflow харуулах байсан тул project-ийг хэт томруулах шаардлагагүй гэж үзсэн. Жижиг project байсан ч planning, architecture, testing, documentation, reflection зэрэг бүх software construction алхмуудыг харуулах боломжтой.

3.2 Stack comparison

Доорх гурван stack-ийг харьцуулсан:

Criteria	Node.js + Express + SQLite	Python + Flask + SQLite	Java Spring Boot + H2/PostgreSQL
----------	----------------------------	-------------------------	----------------------------------

REST API хийх	Express route хурдан, энгийн	Flask жижиг API-д тохиромжтой	Structured боловч setup их
Test хийх	Jest + Supertest тохиромжтой	pytest ашиглаж болно	JUnit хүчтэй боловч config их
Database	SQLite server шаардахгүй	SQLite built-in support сайн	H2/PostgreSQL илүү тохиромжтой
Frontend холболт	JavaScript ecosystem	нэг Frontend JS-тэй хэл ялгаатай	Project томрох магадлалтай
Scope-д нийцэх	Маш сайн	Сайн	Харьцангуй хүнд

Эцэст нь **Node.js + Express + SQLite** stack сонгосон. Сонгосон шалтгаан нь жижиг REST API хийхэд энгийн, Jest болон Supertest ашиглан test бичихэд тохиромжтой, SQLite нь тусдаа database server шаардахгүй, minimal frontend-д plain HTML/CSS/JavaScript хангалттай, мөн security concept-уудыг тайлбарлахад ойлгомжтой байсан.

3.3 Architecture diagram

```

flowchart TD
    User[User Browser] --> Frontend[Minimal Frontend HTML/CSS/JS]
    Frontend --> API[Express REST API]
    API --> Validator[URL and Expiration Validator]
    API --> Service[URL Service]
    Service --> DB[(SQLite Database)]
    User --> ShortURL[Short URL /:code]
    ShortURL --> API
    API --> Redirect[Redirect to Original URL]

```

Рис. 1: Mermaid architecture diagram-ийн эх хувилбар

Architecture-ийг route, validator, service, database гэсэн хэсгүүдэд салгаж бодсон. Route layer нь request/response handling хийнэ. Validator нь URL болон expiration date шалгана. Service layer нь short code үүсгэх, database operation хийх logic-ийг агуулна. Database layer нь SQLite connection болон query-г хариуцна.

3.4 Repository structure

Даалгаврын шаардлагын дагуу дараах repository structure-ийг баримталсан:

```

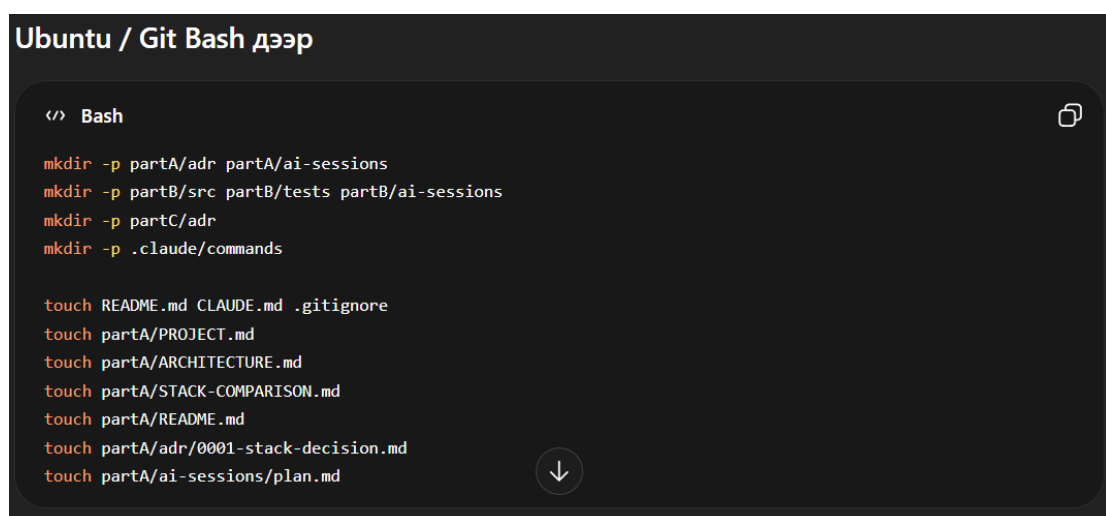
bie-daalt-13/
|-- README.md
|-- CLAUDE.md
|-- .gitignore
|-- .claude/
| '-- commands/

```

```
| |-- review.md
| |-- test.md
| |-- docs.md
| |-- security.md
| '-- commit.md
|-- partA/
| |-- PROJECT.md
| |-- ARCHITECTURE.md
| |-- STACK-COMPARISON.md
| |-- README.md
| |-- adr/
| | '-- 0001-stack-decision.md
| '-- ai-sessions/
| '-- plan.md
|-- partB/
| |-- src/
| |-- tests/
| |-- README.md
| |-- openapi.yaml
| '-- ai-sessions/
'-- partC/
    |-- AI-USAGE-REPORT.md
    |-- adr/
    | '-- 0002-storage-decision.md
    '-- SELF-EVALUATION.md
```

3.5 Setup screenshot evidence

AI assistant project folder үүсгэх command санал болгосон. Энэ нь planning болон setup үе шатанд AI-г ашигласны нотолгоо болсон.



```
Ubuntu / Git Bash дээр

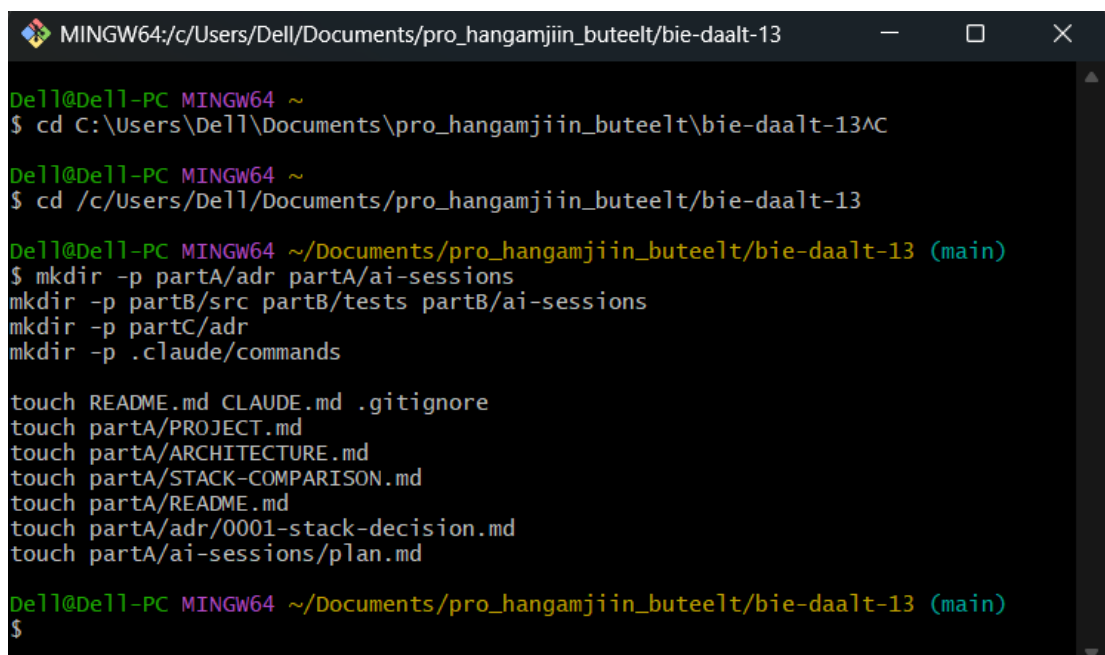
</> Bash

mkdir -p partA/adr partA/ai-sessions
mkdir -p partB/src partB/tests partB/ai-sessions
mkdir -p partC/adr
mkdir -p .claude/commands

touch README.md CLAUDE.md .gitignore
touch partA/PROJECT.md
touch partA/ARCHITECTURE.md
touch partA/STACK-COMPARISON.md
touch partA/README.md
touch partA/adr/0001-stack-decision.md
touch partA/ai-sessions/plan.md
```

Рис. 2: AI project folder creation command санал болгосон screenshot

Дараа нь би Git Bash дээр project folder болон file үүсгэх command ажиллуулсан.



```
MINGW64:/c/Users/Dell/Documents/pro_hangamjiin_buteelt/bie-daalt-13
Dell@Dell-PC MINGW64 ~
$ cd C:\Users\Dell\Documents\pro_hangamjiin_buteelt\bie-daalt-13^C
Dell@Dell-PC MINGW64 ~
$ cd /c/Users/Dell/Documents/pro_hangamjiin_buteelt/bie-daalt-13
Dell@Dell-PC MINGW64 ~/Documents/pro_hangamjiin_buteelt/bie-daalt-13 (main)
$ mkdir -p partA/adr partA/ai-sessions
mkdir -p partB/src partB/tests partB/ai-sessions
mkdir -p partC/adr
mkdir -p .claude/commands
touch README.md CLAUDE.md .gitignore
touch partA/PROJECT.md
touch partA/ARCHITECTURE.md
touch partA/STACK-COMPARISON.md
touch partA/README.md
touch partA/adr/0001-stack-decision.md
touch partA/ai-sessions/plan.md
Dell@Dell-PC MINGW64 ~/Documents/pro_hangamjiin_buteelt/bie-daalt-13 (main)
$
```

Рис. 3: Git Bash дээр project structure үүсгэсэн screenshot

VS Code Explorer дээр required structure үүссэн байдлыг дараах screenshot харуулна.

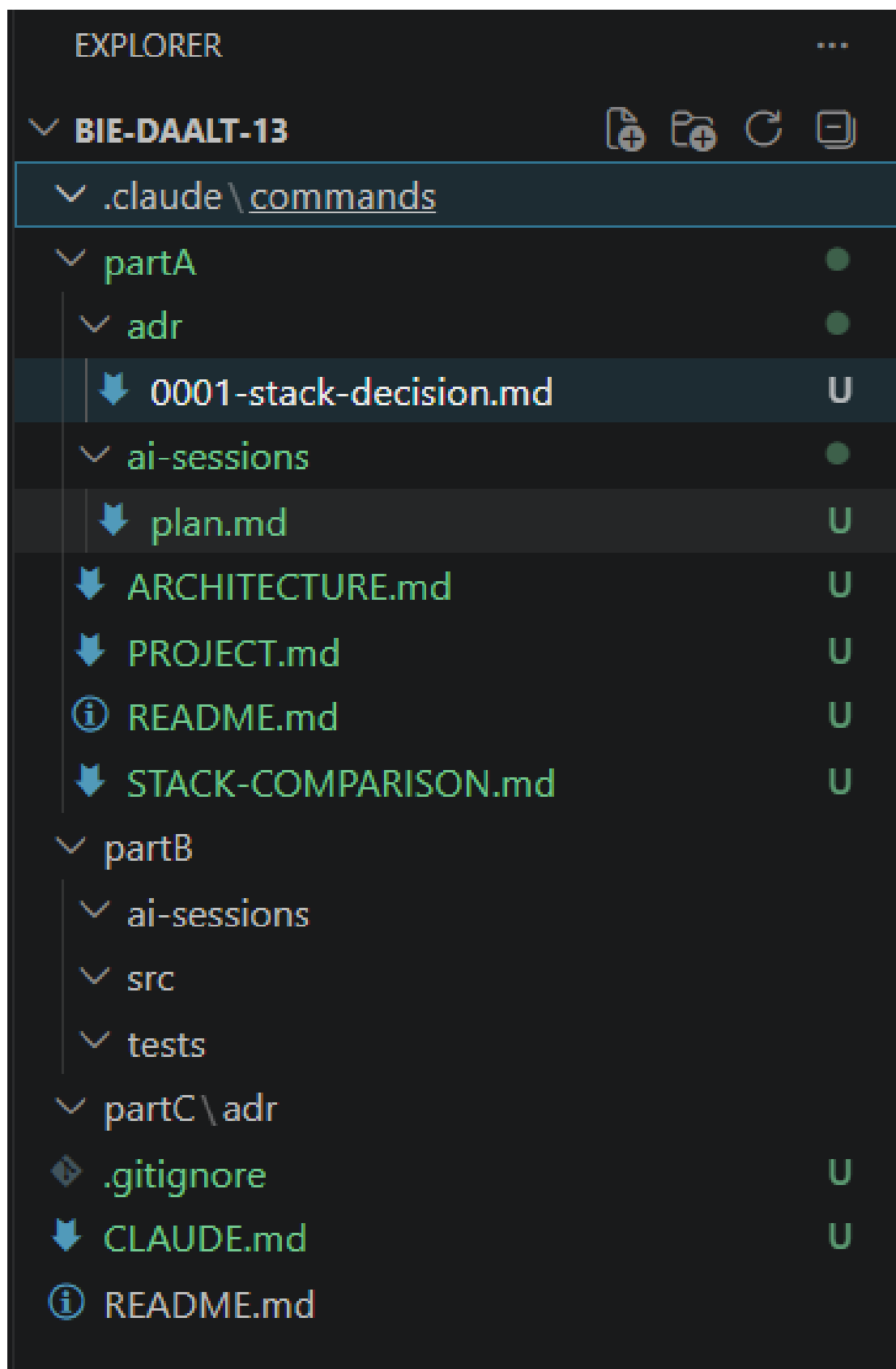


Рис. 4: VS Code дээр project structure үүссэн screenshot

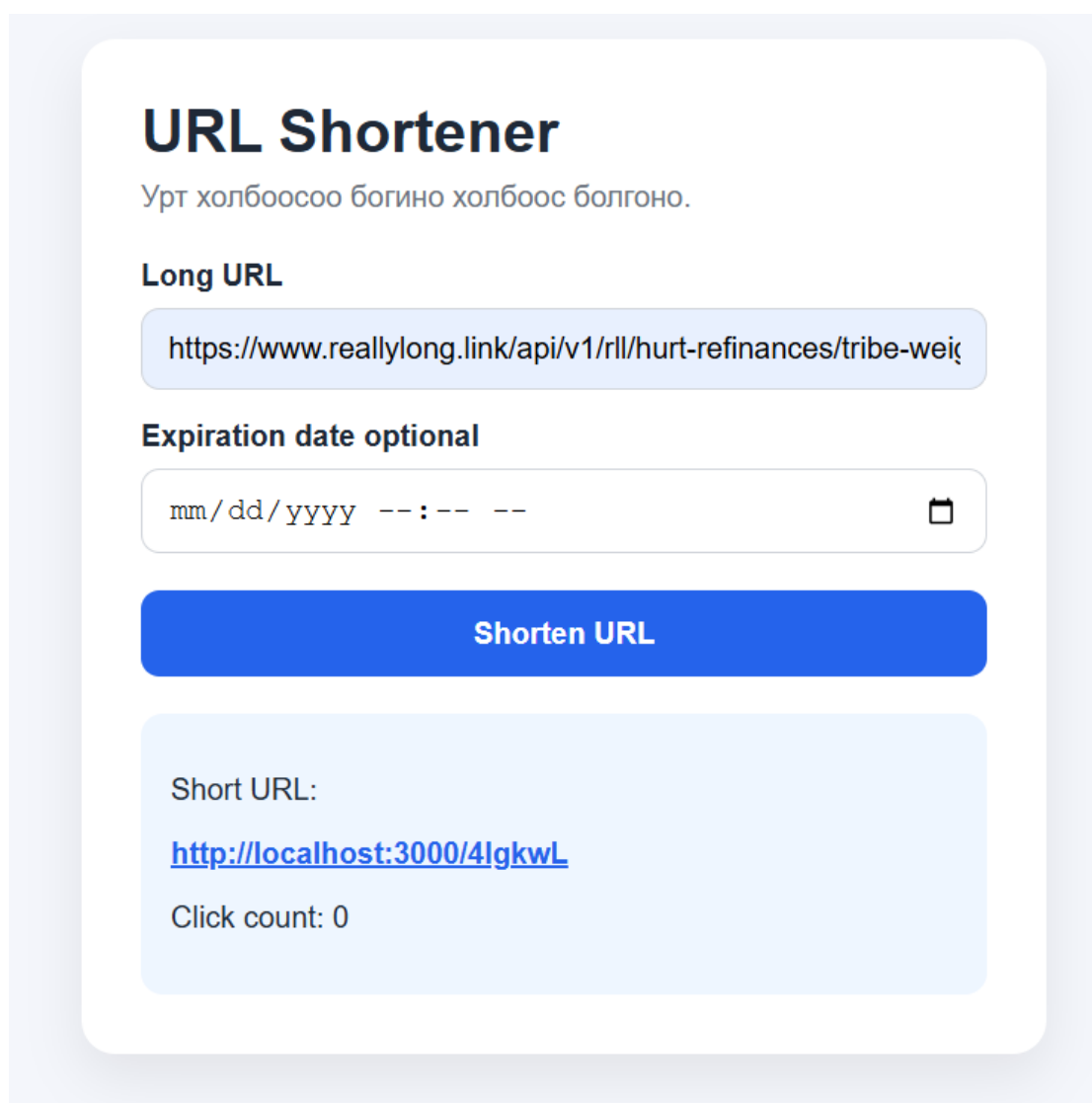
3.6 ADR-001: Stack decision

ADR-001 дээр Node.js + Express + SQLite stack сонгосон шийдвэрийг Accepted гэж тэмдэглэсэн. Context хэсэгт project нь оюутны assignment тул жижиг, ойлгомжтой, test хийх боломжтой байх ёстой гэж тайлбарласан. Alternatives хэсэгт Python + Flask + SQLite болон Java Spring Boot + H2/PostgreSQL хувилбаруудыг авч үзсэн. Consequences хэсэгт Express хурдан, SQLite setup багатай боловч JavaScript runtime error болон SQLite scalability limitation байгаа гэж тэмдэглэсэн.

4 Part B — Build

4.1 Implementation result

Build хэсэгт URL Shortener-ийн үндсэн feature-үүдийг хэрэгжүүлэхээр ажилласан. Frontend дээр хэрэглэгч URL оруулж, backend рүү request илгээж, short URL авах workflow-г minimal хэлбэрээр хийсэн. Доорх screenshot нь frontend ажиллаж, short URL үүссэн байдлыг харуулж байна.



The screenshot shows a web interface for a URL shortener. At the top, the title "URL Shortener" is displayed in a large, bold, dark font. Below it, a subtitle in Mongolian reads "Урт холбоосоо богино холбоос болгоно." (Convert long links into short links). The interface includes a section labeled "Long URL" with a text input field containing a long, complex URL. Below this is an "Expiration date optional" section with a date input field showing "mm/dd/yyyy --:-- --" and a calendar icon. A prominent blue button labeled "Shorten URL" is positioned below the date field. The bottom section, highlighted with a light blue background, displays the "Short URL:" as <http://localhost:3000/4lgkwL> and the "Click count: 0".

Рис. 5: URL Shortener frontend ажиллаж байгаа screenshot

Мөн Part B-ийн бодит source code, frontend, test, OpenAPI болон AI session log файлууд VS Code Explorer дээр дараах байдлаар харагдсан. Энэ нь build хэсгийн repository structure шаардлагын дагуу бүрдсэнийг нотолж байна.

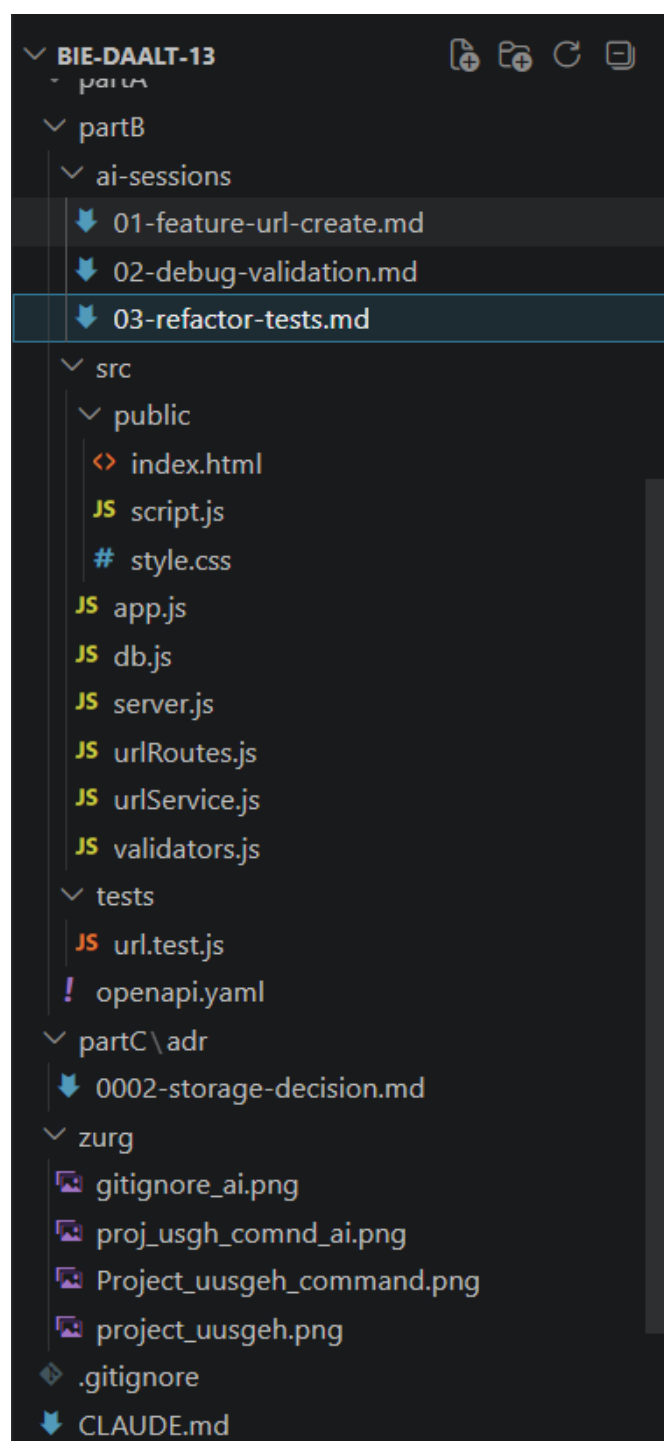


Рис. 6: Part B source code, tests, OpenAPI болон AI session log бүтэц

4.2 API endpoint-үүд

Method	Endpoint	Зорилго
POST	/api/shorten	Long URL болон optional expiration date авч short URL үүсгэнэ.

GET	/:code	Short code-oor original URL руу redirect хийнэ.
GET	/api/stats/:code	Short URL-ийн statistics буцаана.
GET	/health	Server ажиллаж байгаа эсэхийг шалгах зориулалттай.

4.3 Test plan

Test-үүдийг Jest + Supertest ашиглан ажиллуулах зориулалттай. Даалгаврын шаардлага 10-аас дээш test байсан тул дараах test case-үүдийг төлөвлөсөн:

1. Valid long URL өгөхөд 201 болон short code буцаах.
2. URL хоосон өгөхөд 400 validation error буцаах.
3. ftp:// protocol өгөхөд 400 validation error буцаах.
4. Invalid URL string өгөхөд 400 error буцаах.
5. Valid expiration date өгөхөд URL үүсэх.
6. Past expiration date өгөхөд 400 error буцаах.
7. Existing short code redirect хийхэд 302 response өгөх.
8. Not found short code дээр 404 response өгөх.
9. Redirect хийсний дараа click count нэмэгдэх.
10. Stats endpoint valid code дээр metadata буцаах.
11. Stats endpoint invalid code дээр 404 response өгөх.
12. Expired URL redirect хийхэд expired response буцаах.

Энд “бүх test pass болсон” гэж бичээгүй. Учир нь test-ийн бодит үр дүнг `npm test` command ажиллуулсны дараа terminal output-оор тэмдэглэх ёстой.

4.4 OpenAPI 3.0

API project тул OpenAPI 3.0 YAML specification бэлдсэн. Үүнд `POST /api/shorten`, `GET /:code`, `GET /api/stats/:code`, `GET /health` endpoint-үүдийн request болон response schema-г тодорхойлсон. OpenAPI ашигласнаар API contract нь code-оос тусдаа documentation хэлбэрээр хадгалагдана.

4.5 Custom slash commands

`.claude/commands` дотор дараах custom command-уудыг төлөвлөсөн:

Command	Зорилго
<code>/review</code>	Code болон documentation review checklist rapрах
<code>/test</code>	Test idea болон coverage шалгах
<code>/docs</code>	README, ADR, AI log бичвэр сайжруулах
<code>/security</code>	URL validation, SQL injection, redirect risk шалгах
<code>/commit</code>	Conventional commit message санал болгох

4.6 Commit workflow

Commit message-ийг conventional commit style ашиглан бичихээр төлөвлөсөн. Жишээ нь `feat: add url shortening endpoint`, `fix: validate unsupported url protocols`, `test: add redirect tests`, `docs: add ai usage report` гэх мэт. AI-assisted commit дээр Co-Authored-By мөр ашигласан гэдгийг ил тод дурдсан.

5 Part C — Reflection

5.1 AI-г юунд ашигласан, юуг өөрөө хийсэн бэ?

AI assistant-ийг project-ийн planning, structure rapрах, stack comparison draft бичих, architecture diagram-ийн санаа авах, test checklist rapрах, security checklist боловсруулах, documentation draft бэлтгэхэд ашигласан. Жишээ нь AI `.gitignore` дээр `node_modules/`, `.env`, runtime database file зэрэг зүйлсийг ignore хийх хэрэгтэй гэж санал болгосон.

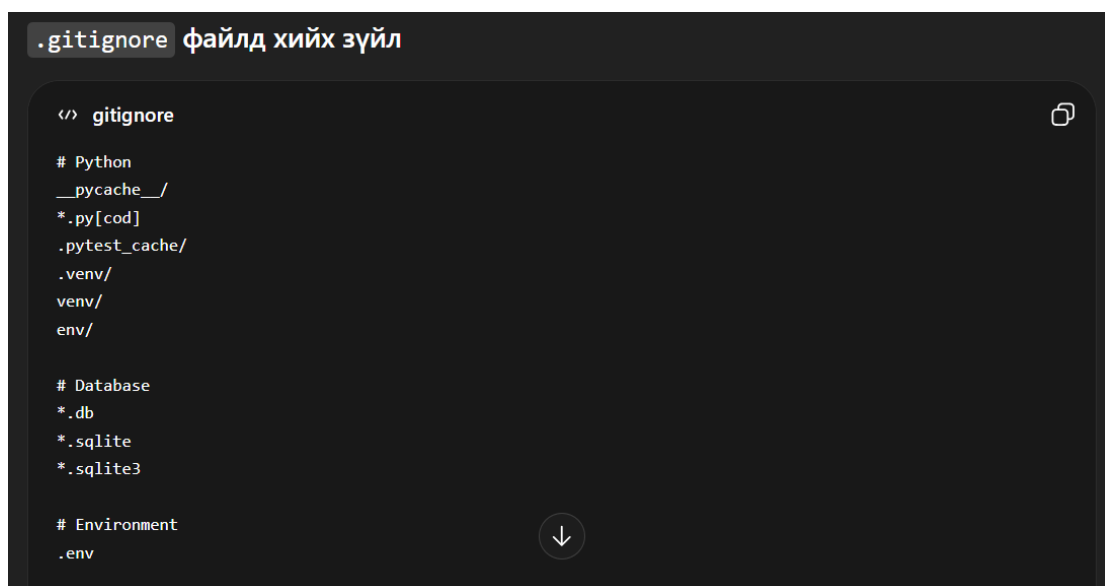


Рис. 7: AI `.gitignore` ignore rule санал болгосон screenshot

Гэхдээ AI-ийн өгсөн suggestion-ийг шууд final гэж үзээгүй. Би repository structure-ийг өөрөө шалгасан, Git Bash дээр command ажиллуулсан, VS Code дээр folder structure үүссэн эсэхийг харсан, screenshot evidence авсан, terminal error уншсан, Git status шалгасан. Stack сонголтын final decision, scope-д юу оруулах, юу оруулахгүй байх, documentation дээр ямар claim бичихгүй байх зэрэг шийдвэрийг хүний review-тэй гаргасан.

Өөрөөр хэлбэл AI нь “бэлэн project хийж өгсөн” биш, харин туслах role-той байсан. Миний хувьд AI-г planning partner, reviewer, checklist generator, documentation draft assistant байдлаар ашигласан. Харин project-ийн бодит явц, screenshot evidence, алдаа засах, final тайлбар бичих ажлыг өөрөө хийсэн.

5.2 Hallucination жишээ ба зассан арга

Жишээ 1: server.js файл байгаа мэт таамагласан. AI заримдаа Node.js project-ийн нийтлэг structure гэж үзээд `server.js` entry file байгаа мэтээр зөвлөсөн. Гэтэл `npm start` ажиллуулах үед `server.js` файл байхгүйгээс алдаа гарсан. Би энэ үед AI-ийн үгэнд шууд итгэлгүй, terminal error message болон `package.json` script-ийг шалгасан. Ингэснээр entry file дутуу эсвэл path зөрүүтэй байгааг олсон. Засахдаа `partB/src/server.js` үүсгэх эсвэл `package.json` script-ийг бодит entry file рүү заах шаардлагатай гэж дүгнэсэн.

Жишээ 2: node_modules/.package-lock.json tracking-д орсон. AI `.gitignore` дээр `node_modules/` ignore хийх зөв suggestion өгсөн ч аль хэдийн Git tracking-д орсон file автоматаар салдаггүй. Би `git status` ашиглан `node_modules/.package-lock.json` tracking-д орсон асуудлыг олсон. Дараа нь `.gitignore` шалгаж, шаардлагатай үед `git rm -cached` ашиглан Git index-ээс салгах ёстой гэж ойлгосон.

Жишээ 3: Test pass claim хийх эрсдэл. AI documentation draft бичих үед “tests pass” гэх ерөнхий өгүүлбэр оруулах магадлалтай. Гэхдээ бодитоор test ажиллуулаагүй бол ийм claim бичих нь буруу. Тиймээс би тайландаа test-ийг `npm test` command-аар ажиллуулах зориулалттай, үр дүнг ажиллуулсны дараа тэмдэглэх ёстой гэж бичсэн.

5.3 Security болон license-ийн анхаарал

URL Shortener project-ийн хамгийн чухал security асуудал нь redirect behavior юм. Хэрэглэгчийн өгсөн URL-ийг шалгахгүйгээр redirect хийх нь эрсдэлтэй. Тиймээс validation дээр зөвхөн `http://` болон `https://` protocol зөвшөөрөхөөр төлөвлөсөн. Энэ нь `ftp://`, `file://`, `javascript:` зэрэг unsupported эсвэл эрсдэлтэй protocol-ийг барасгана.

Мөн SQL injection эрсдэлийг барасгахын тулд parameterized query ашиглах шаардлагатай гэж тайлбарласан. Хэрэглэгчийн оруулсан URL болон code-г SQL string concatenation хийж query-д шууд оруулахгүй байх ёстой. Error response дээр database-ийн дотоод алдаа, stack trace, file path зэргийг хэрэглэгчид ил гаргахгүй байх нь зөв.

License талаас dependency цөөн байлгахыг зорьсон. `node_modules/` folder-ийг Git-д commit

хийхгүй байх, `.env` болон `runtime database file`-ийг `repository`-д оруулахгүй байх, гуравдагч талын `code block`-ийг эх сурвалжгүйгээр хуулж ашиглахгүй байхыг анхаарсан.

5.4 AI юуг хурдан болгосон бэ?

AI planning-ийг хурдан болгосон. `Repository structure`, `README outline`, `ADR structure`, `session log format`, `stack comparison table` зэрэг нь эхний draft байдлаар хурдан гарсан. Хэрэв бүх зүйлийг хоосон файлаас эхэлсэн бол илүү их цаг шаардах байсан.

Мөн AI test idea гаргахад тус болсон. `Valid URL`, `invalid URL`, `unsupported protocol`, `expiration date`, `redirect`, `not found`, `stats`, `click count` зэрэг case-үүдийг `checklist` байдлаар хурдан жагсаасан. `Documentation` дээр ч AI heading structure гаргаж өгснөөр би өөрийн project-ийн бодит явц, `screenshot evidence`, алдаа зассан арга дээр төвлөрөх боломжтой болсон.

`Security checklist` дээр AI `URL validation`, `SQL injection`, `.env`, `node_modules`, `error response` зэрэг зүйлсийг сануулсан. Би тэдгээрийг project-ийн `score`-д тааруулж хэрэглэсэн.

5.5 AI юуг удаан болгосон бэ?

AI-ийн санал бүрийг шалгах шаардлагатай байсан нь зарим үед удаан болгосон. Жишээ нь `server.js` байгаа мэт таамагласан асуудлыг засахад `terminal output` унших, `file structure` шалгах хэрэгтэй болсон. Мөн AI-ийн бичвэр заримдаа хэт ерөнхий эсвэл хэт төгс сонсогддог тул Монгол хэл дээр бодит оюутны тайлан шиг болгож засах шаардлагатай болсон.

AI заримдаа `scope` томруулах санаа өгдөг. `Authentication`, `dashboard`, `deployment`, `analytics` зэрэг feature-үүд хэрэгтэй мэт санагдаж болох ч энэ даалгаврын `score`-д илүүц байсан. Ийм саналуудыг хасаж, project-ийг жижиг боловч `requirement` хангах хэмжээнд барих хэрэгтэй болсон.

5.6 Skill atrophy эрсдэлийг хэрхэн зохицуулсан бэ?

AI-д хэт найдаад өөрийн чадвар сулрах эрсдэлийг багасгахын тулд би AI-ийн хариуг шууд `final` гэж үзээгүй. `Error message`-г өөрөө уншсан, `Git status` шалгасан, `project structure`-ийг өөрөө харсан, `screenshot evidence` өөрөө бүрдүүлсэн. `Documentation`-ийг өөрийн үгээр Монгол хэлээр бичих нь тухайн feature болон `decision`-ийг үнэхээр ойлгосон эсэхээ шалгах нэг арга болсон.

Мөн `test plan`-ийн `test` бүр яагаад хэрэгтэйг бодож бичсэн. Жишээ нь `invalid protocol test` нь `security`-тэй холбоотой, `click count test` нь `redirect side effect` шалгана, `stats endpoint test` нь `database query` болон `response shape` шалгана. Ийм байдлаар AI-аас зөвхөн `list` аваад зогсохгүй, өөрөө `reasoning` хийсэн.

6 ADR-002: Storage decision

Build-ийн явцад URL mapping, click count, created date, expiration date зэргийг хаана хадгалах вэ гэсэн architecture decision гарсан. In-memory object ашиглавал хурдан demo хийх боломжтой боловч server restart хийхэд data устна. JSON file ашиглаж болох ч update хийх, concurrent write зохицуулахад эвгүй. PostgreSQL production-д илүү тохиромжтой боловч оюутны project-д setup ихтэй.

Иймээс SQLite ашиглахаар шийдсэн. SQLite нь тусдаа database server шаардахгүй, local development-д энгийн, click count болон statistics query хийхэд хангалттай. Сөрөг тал нь том traffic болон distributed deployment-д тохиромжгүй. Гэхдээ энэ assignment-ийн scope-д хамгийн зөв тэнцвэртэй сонголт гэж үзсэн.

7 Өөрийн үнэлгээ

7.1 Хэрэв шалгалт өнөөдөр болбол би энэ кодыг өөрөө бичиж чадах уу?

Тийм, үндсэн хувилбарыг өөрөө бичиж чадна гэж бодож байна. Express дээр route үүсгэх, request body авах, URL validation хийх, SQLite-д хадгалах, redirect response буцаах зэрэг гол хэсгүүдийг ойлгосон. Гэхдээ AI-гүйгээр яг хурдан, цэвэрхэн structure-тэй бичихэд илүү их цаг авах байх.

7.2 Дахин хийнэ гэвэл юуг өөрөөр хийх вэ?

Дахин хийх бол repository structure болон package.json script-ийг бүр эхэнд нь зөв тохируулна. Мөн .gitignore-ийг project эхлэх үед шууд сайн бичиж, node_modules tracking-д оруулахгүй байх байсан. Test-ийг feature бичихтэй зэрэгцүүлэн нэмбэл илүү найдвартай байх байсан.

7.3 Энэ туршлагаас юу сурсан бэ?

AI-г зөв ашиглахын тулд prompt сайн бичихээс гадна AI-ийн хариуг шалгах чадвар илүү чухал гэдгийг ойлгосон. AI хурдан draft өгдөг боловч file үнэхээр байгаа эсэх, command ажиллах эсэх, Git tracking зөв эсэх, test pass эсэхийг хүн өөрөө шалгах ёстой. AI-г орондоо код бичүүлдэг хэрэгсэл биш, харин planning, review, checklist, documentation draft гаргадаг туслах гэж ашиглах нь илүү зөв гэж дүгнэсэн.

8 Дүгнэлт

Энэ project нь жижиг хэмжээний URL Shortener боловч AI-assisted software construction-ийн үндсэн алхмуудыг харуулахад тохиромжтой байсан. Part A дээр planning, stack comparison, architecture, ADR-001 хийсэн. Part B дээр URL shortening, redirect, click count, expiration validation, stats endpoint, frontend, OpenAPI, test plan, custom command workflow тайлбарласан. Part C дээр AI usage reflection, hallucination жишээ, security/license анхаарал, storage decision, self-evaluation бичсэн.

AI project-ийг хурдан эхлүүлэх, documentation болон checklist гаргахад маш хэрэгтэй байсан. Гэхдээ AI-ийн output-ийг human review хийхгүйгээр шууд ашиглах нь алдаа гаргах эрсдэлтэй. Энэ даалгавраас авсан хамгийн гол сургамж бол AI болон хүний review хоёрыг зөв хослуулах хэрэгтэй гэсэн ойлголт юм.